

REFLECTION AT THE REWRITE LAYER: A CATEGORICAL FRAMEWORK FOR
SOURCE-EXTENSIBLE VERIFIED TRANSFORMATION IN DATAFLOW IRS

Isaac H. Lopez Díaz

University of Puerto Rico Río Piedras

2026

Professor

Tatiana Castro Velez, Faculty of Natural Sciences, Department of Computer Science

ABSTRACT

REFLECTION AT THE REWRITE LAYER: A CATEGORICAL FRAMEWORK FOR
SOURCE-EXTENSIBLE VERIFIED TRANSFORMATION IN DATAFLOW IRS

Isaac H. Lopez Díaz

Tatiana Castro Velez

TABLE OF CONTENTS

ABSTRACT	ii
CHAPTER 1 : Introduction	1
CHAPTER 2 : Theoretical Foundations	2
CHAPTER 3 : Literature Review	3
CHAPTER 4 : Analysis	4
CHAPTER 5 : New Proposal	5
5.1 How would you use the existing literature to design a new solution?	5
5.2 What techniques will you combine?	5
5.3 What limitations will you resolve?	5
5.4 What conceptual architecture would it have?	5
5.5 What kind of evaluation would it need?	5
CHAPTER 6 : Conclusion	7
BIBLIOGRAPHY	8

CHAPTER 1

Introduction

CHAPTER 2

Theoretical Foundations

CHAPTER 3

Literature Review

CHAPTER 4

Analysis

CHAPTER 5

New Proposal

- 5.1. How would you use the existing literature to design a new solution?
- 5.2. What techniques will you combine?
- 5.3. What limitations will you resolve?
- 5.4. What conceptual architecture would it have?
- 5.5. What kind of evaluation would it need?

Conceptual evaluation: the algebra-coverage property is typechecked; the adequacy check catches a sample of disagreements on test inputs; the worked example (loops + mutable state) demonstrates the system handles a construct outside the base translation. Comparative evaluation: against TorchDynamo (translation is sound by check, not opportunistic), against TensorRight (translation extension rather than graph rewrite), against MLIR (reflective rather than statically declared extensions). Future evaluation: once mechanized, the adequacy theorem replaces sampling with a proof. Stating this as future evaluation is honest and shows you understand the trajectory.

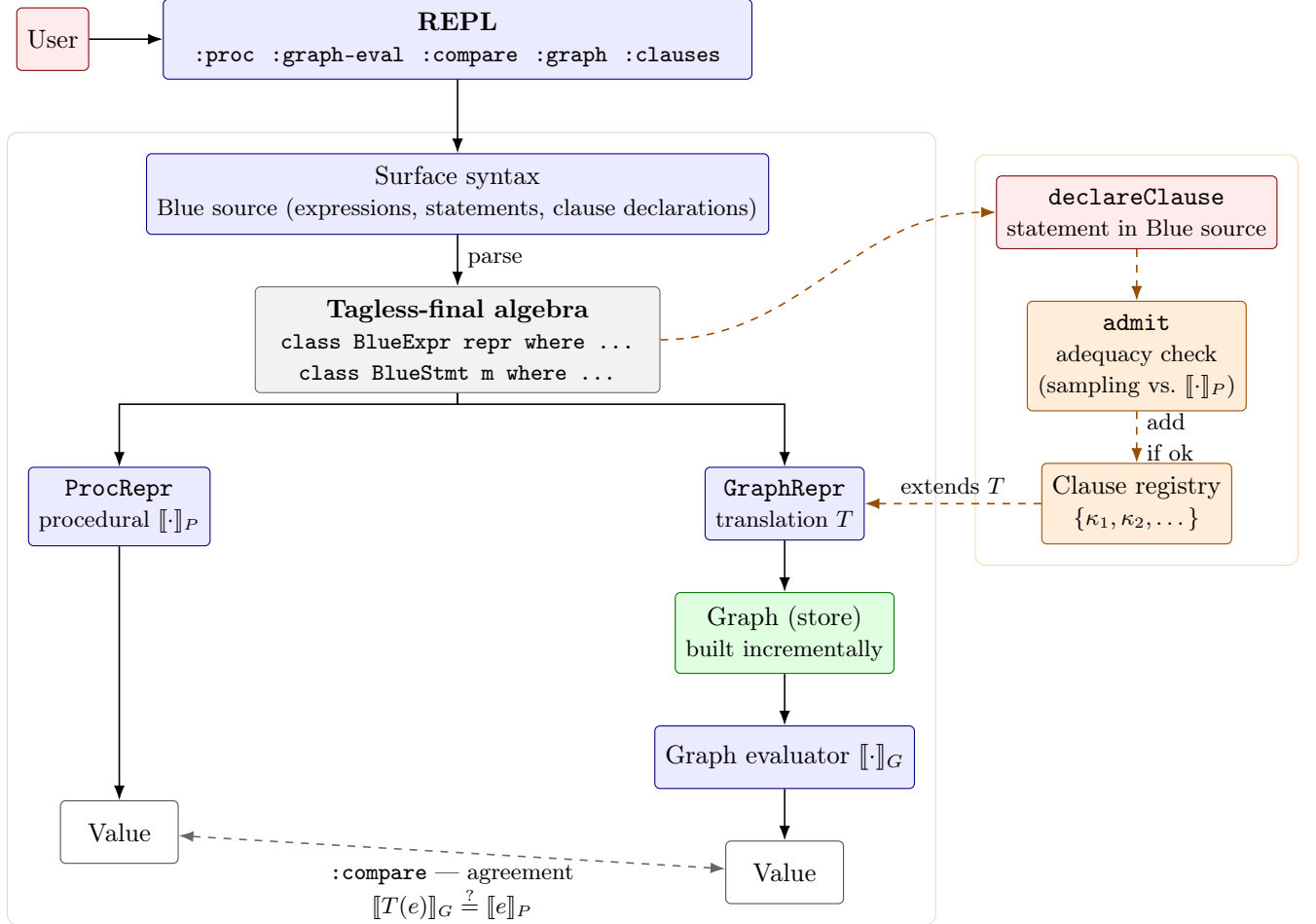


Figure 5.1: Conceptual architecture of Blue. A Blue program is parsed into a tagless-final algebra with two interpretations: a procedural interpretation **ProcRepr** realizing $\llbracket \cdot \rrbracket_P$, and a translation interpretation **GraphRepr** realizing T , which builds a graph that serves as the runtime store. The graph is evaluated via $\llbracket \cdot \rrbracket_G$. A Blue source program may also include **declareClause** statements which install new translation clauses; each clause is checked for adequacy against $\llbracket \cdot \rrbracket_P$ on admission and, if accepted, extends T for subsequent translation. The **:compare** command in the REPL evaluates a program under both interpretations and reports whether they agree, giving the user a runtime witness of the adequacy property the system maintains.

CHAPTER 6

Conclusion

BIBLIOGRAPHY

- Nada Amin and Tiark Rompf. Collapsing towers of interpreters. *Proceedings of the ACM on Programming Languages*, 2(POPL):1–33, 2018. doi: 10.1145/3158140.
- Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarakar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, pages 929–947, New York, NY, USA, 2024. Association for Computing Machinery. doi: 10.1145/3620665.3640366.
- Jai Arora, Sirui Lu, Devansh Jain, Tianfan Xu, Farzin Houshmand, Phitchaya Mangpo Phothilimthana, Mohsen Lesani, Praveen Narayanan, Karthik Srinivasa Murthy, Rastislav Bodík, Amit Sabne, and Charith Mendis. TensorRight: Automated verification of tensor graph rewrites. *Proceedings of the ACM on Programming Languages*, 9(POPL):832–863, 2025. doi: 10.1145/3704865.
- Kenichi Asai. Compiling a reflective language using MetaOCaml. In *Proceedings of the 2014 International Conference on Generative Programming: Concepts and Experiences*, GPCE '14, pages 113–122, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2658761.2658775.
- Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages. *Journal of Functional Programming*, 19(5):509–543, 2009. doi: 10.1017/S0956796809007205.
- Conal Elliott. Compiling to categories. *Proceedings of the ACM on Programming Languages*, 1(ICFP):1–27, 2017. doi: 10.1145/3110271.
- Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi: 10.1145/1538788.1538814.
- Amanda Liu, Gilbert Louis Bernstein, Adam Chlipala, and Jonathan Ragan-Kelley. Verified tensor-program optimization via high-level scheduling rewrites. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–28, 2022. doi: 10.1145/3498717.
- Brian Cantwell Smith. Reflection and semantics in LISP. In *Proceedings of the 11th ACM SIGACT-*

SIGPLAN Symposium on Principles of Programming Languages, POPL '84, pages 23–35, New York, NY, USA, 1984. Association for Computing Machinery. doi: 10.1145/800017.800513.

Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, 2021. doi: 10.1145/3434304.